

28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)

Open-source Python repository for data drift analysis

Krzysztof Wrobel^{a,*}, Piotr Porwik^a, Tomasz Orczyk^a

^a*Institute of Computer Science, University of Silesia in Katowice, Bedzinska 39, 41-200 Sosnowiec, Poland*

Abstract

In this paper, we propose practical Python programs together with appropriate environments for the analysis of data stream drift, including the analysis of feature drift. The proposed approach contains a description of both, synthetic and real datasets. These datasets include different types of drifts like sudden, incremental, or gradual. Also, the location of the drift can be programmed. Additionally, we propose Java scripts that allow specifying the number and place of drift locations. The software is focused on prequential error methodology.

Our proposition can be used by scientists interested in machine learning and concept drift detection because the proposed solution makes it easier to conduct practical experiments on that matter. The proposed solution allows for conducting experiments in a homogeneous programming environment. Versions of Python programs, stored in the GitHub repository, contain implementations of popular classifiers and drift detectors. The GitHub repository is located in the Public Domain.

© 2024 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the 28th International Conference on Knowledge Based and Intelligent information and Engineering Systems

Keywords: Data drift; Classification; Drift detection

1. Introduction

Research on different drift forms is conducted in multiple programming environments with various libraries dedicated to machine learning purposes. This often leads to different results even for the same reference data. This reduces the reliability of the research and introduces uncertainty in the interpretation of the obtained results. The proposed solution alleviates these problems by providing a homogeneous, open-source environment.

Traditional classification methods, where the source data are stationary, assume data are collected before the learning process [2, 5, 14, 16]. For such assumptions, single and ensemble classifiers obtain good accuracy, and their choice depends mostly on individual data characteristics. Unfortunately, nowadays, the amount of data and features used in machine learning and data mining is constantly increasing. High-dimensional data are often Redundant and prone to change, making learning models more sophisticated and time-consuming. Such a phenomenon occurs in non-stationary data, where information comes as a data stream, and data distribution is dynamic over a given period,

* Corresponding author.

E-mail address: krzysztof.wrobel@us.edu.pl

known as concept, data, or feature drift. There may be changes in the transmitted stream in the form of concept or data (feature) drift. Both types of drifts can lead to a decline in the performance of a machine learning model. Therefore, there is also a problem with the training of a classifier [3, 6, 10, 11]. This is an unfavorable phenomenon because, with a changing data stream, the efficiency of each classifier is volatile which must be counteracted. Classification is widely employed in data mining and machine learning. In the traditional machine learning approach, concepts come from a static dataset in which instances fall within a particular statistical distribution. However, this model is unacceptable for many real-world scenarios, including medical data, biometric data, spam, fraud detection, observation of climate change, etc.

Dynamic data analysis is complex, especially when the data stream changes over time. Monitoring and detecting these changes to maintain the learning algorithm's performance is challenging due to the different types of changes in data over time. Due to the non-stationarity of data, the use of techniques to detect changes in data distribution is necessary. Machine learning algorithms should quickly adapt to data evolution and retrain classifiers only when necessary [1, 2, 7, 15].

In classification, samples are provided as a continuous stream of data that is not always strictly stationary. Data of the stream $DS \in R^d$ can be described as an ordered sequence of data of dimension d over time $DS = (x_1, \dots, x_t)$, where x_t is the last recent sample received. Each sample x_i is associated with a label $y_i \in \{1, \dots, l\}$. The data structured this way are saved in *.arff files and subjected to further analysis.

2. Software preparation

The repository consists of five integrated modules responsible for (1) preparation of the synthetic data streams using batch files, (2) drift detector methods, (3) classification algorithms, (4) evaluation metrics with and without drift detection, and (5) graphics display module. Except for module (1), an external part of the project, the rest are integrated into one Python program.

2.1. Synthetic data stream generators

The basic assumption is the ability to generate repeatable data with a fixed number of samples, several drifts, and a specified noise level. The most frequently observed nature of data streams is the rate of change and distribution of data over time. These changes can be sudden, gradual, incremental, and recurring.

Sudden (abrupt) changes This type of drift occurs when data are changed suddenly over time, meaning that another distribution replaces the current distribution [2, 4, 8]. This phenomenon is presented graphically in Fig. 1 for 2-dimensional data. A data file containing sudden drift can be generated using a MOA tool:

```
java -cp moa.jar moa.DoTask "WriteStreamToARFFFile -s (ConceptDriftStream -s (generators.HyperplaneGenerator -a 20) -d (generators.HyperplaneGenerator -i 10 -k 2 -a 20 -n 1 -t 0.5) -p 50000 -w 1) -f Abrupt.arff -m 100000"
```

This allows the drift parameters to be programmed: number of samples (m), number of features (a), and number of attributes with drift (k). Parameter (w) defines the width of concept drift change, (d) indicates drift stream, (m) projects the maximum number of instances, location of the drift (p), the noise level in % (n), and name of the target file (f).

The parameter (s) indicates that the command configures the stream or probability that the change direction is reversed. Parameter (i) denotes seed for random generation of instances and (t) defines the magnitude of change.

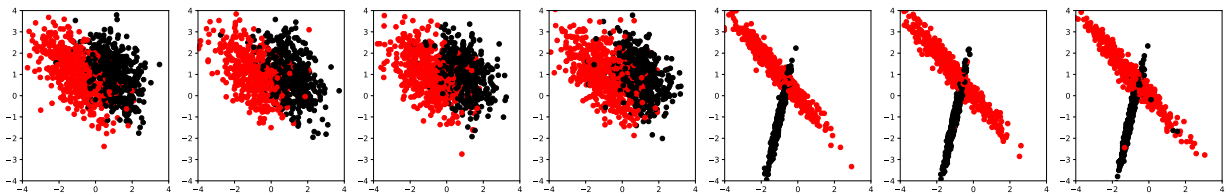


Fig. 1: Example data sequence with sudden changes.

Gradual Drift Unlike sudden drifts, gradual drift involves a slower data change rate over time [2, 8]. This type of drift is more challenging to detect. In this type of drift, the probability of obtaining samples from the first concept decreases, and the likelihood of getting samples from the next concept increases. The challenge is that the timing of these changes must be determined. An illustration of a gradual drift is shown in Fig. 2. Gradual drift can be generated using a MOA tool:

```
java -cp moa.jar moa.DoTask "WriteStreamToARFFFile -s (ConceptDriftStream -s (generators.HyperplaneGenerator -a 20) -d (generators.HyperplaneGenerator -i 10 -k 2 -a 20 -n 1 -t 0.5) -p 50000 -w 20000) -f Gradual.arff -m 100000"
```

Observing the values of the parameters p and w , it can be seen that the drift is now more dispersed.

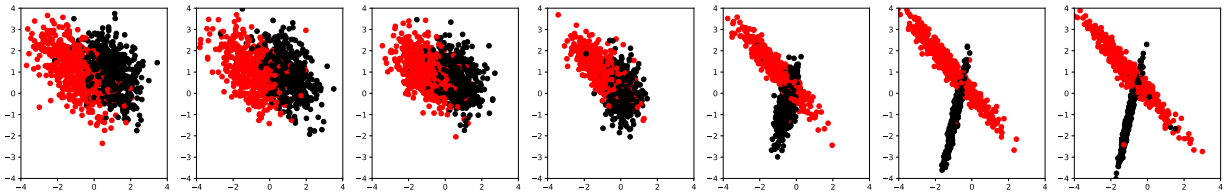


Fig. 2: Example data sequence with gradual changes.

Incremental Drift Incremental drift occurs when a series of barely prolonged incremental changes to the data appear. For this reason, drift may only be recognized after some time [2, 8]. An illustration of the incremental drift is shown in Fig. 3. As before, artificial incremental drift can be defined in a MOA tool:

```
java -cp moa.jar moa.DoTask "WriteStreamToARFFFile -s (ConceptDriftStream -s (generators.HyperplaneGenerator -i 10 -k 2 -a 20 -n 1 -t 0.01 -s 10) -f Incremental.arff -m 100000"
```

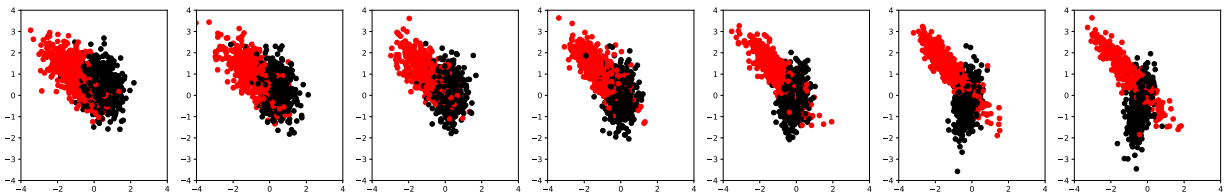


Fig. 3: Example data sequence with incremental changes.

Recurring Drift In this kind of drift, the concept reappears after some period. It means we observe a frequent change in concept in the stream [2, 8, 13]. These types of changes are shown in Fig. 4. Different types of such drift can be configured in the appropriate MOA script:

```
java -cp moa.jar moa.DoTask "WriteStreamToARFFFile -s (ConceptDriftStream -s (generators.HyperplaneGenerator -a 20) -d (ConceptDriftStream -s (generators.HyperplaneGenerator -i 10 -a 20 -n 1 -t 0.5) -d (ConceptDriftStream -s (generators.HyperplaneGenerator -a 20) -d (generators.HyperplaneGenerator -i 10 -a 20 -n 1 -t 0.5) -p 25000 -w 1) -p 25000 -w 1) -f Recurring.arff -m 100000"
```

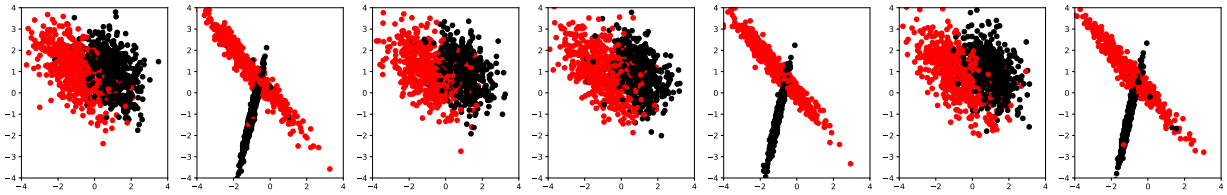


Fig. 4: Example data sequence with recurring abrupt changes.

The input file in ARFF format (for example, spam.arff) is loaded using the procedure:

```
from scipy.io import arff
from sklearn.preprocessing import LabelEncoder
def loadData(fileName):
    classLabelEncoder = LabelEncoder()
    dataFrame = pd.DataFrame(arff.loadarff(fileName)[0])
    for i in range(0, len(dataFrame.columns)):
        col = classLabelEncoder.fit_transform(dataFrame.iloc[:, i].values)
        dataFrame.iloc[:, i] = col
    return dataFrame

fileName = "Spam.arff"
dataFrame = loadData(fileName)
```

In the Python-based application, we include a set of commonly used, state-of-the-art drift detectors and classifiers that can be re-trained after the detector has detected drift. We have used Python 3.8, Numpy (v1.23.5), Pandas (v1.5.3), scikit-learn (v1.3.0), and Frouros (v0.6.1) libraries. There is also a version of the programs for Python 3.11 in the repository.

2.2. Classifiers

The classifier assesses the data. If the data is non-stationary, the classification quality decreases when drift occurs, as shown later. To prevent this phenomenon, the classifier should receive a signal that it should be retrained on new data. Therefore, there is a need for cooperation between the classifier and the drift detector. Various classifiers often work based on different rules. The proposed library allows the selection of many of them, as shown below.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.neural_network import MLPClassifier
```

2.3. Drift detectors

Conventional machine learning data analysis operates under the premise that both training and test data maintain a consistent form and share identical distributions [9]. In adhering to this assumption, traditional machine learning methods are applied. In scenarios where the input data exhibits a non-stationary nature, the initial instances from the data stream are utilized to construct a training model capable of predicting target values. Subsequently, the succeeding

phase is monitoring the presence of concept drift in successive data samples. No intervention is warranted if no drift is observed in the data samples. However, the classifier should be re-trained upon detecting a drift using new (current) data instances. Classifiers must be compatible with Python's scikit-learn library API. Frouros library implements modern drift detectors with default parameters stored in config files.

```
# Concept drift / Streaming / Change detection
from frouros.detectors.concept_drift import BOCD, BOCDConfig
from frouros.detectors.concept_drift import CUSUM, CUSUMConfig
from frouros.detectors.concept_drift import PageHinkley, PageHinkleyConfig

# Concept drift / Streaming / Statistical process control
from frouros.detectors.concept_drift import DDM, DDMConfig
from frouros.detectors.concept_drift import ECDDWT, ECDDWTConfig
from frouros.detectors.concept_drift import EDDM, EDDMConfig
from frouros.detectors.concept_drift import HDDMA, HDDMAConfig
from frouros.detectors.concept_drift import HDDMW, HDDMWConfig
from frouros.detectors.concept_drift import RDDM, RDDMConfig

# Concept drift / Streaming / Window based
from frouros.detectors.concept_drift import ADWIN, ADWINConfig
from frouros.detectors.concept_drift import KSWIN, KSWINConfig
from frouros.detectors.concept_drift import STEPDP, STEPDPConfig
```

3. Experiments

3.1. Experimental setup

The main skeleton of the experiment is shown below. In this scheme, the earlier-mentioned definitions are applied. All necessary details of the scheme, especially the function AccuracyWithDrift(), can be found in the GitHub repository (<https://github.com/ZSKPP/feature-drift/>).

```
fileName = "Spam.arff"
dataFrame = loadData(fileName)
trainingSamples = 500
detector = DDM(config = DDMConfig())
clf = RandomForestClassifier(random_state = 0)
accuracyWithoutDrifts =
    AccuracyWithoutDrifts(clf, dataFrame, trainingSamples)
(accuracyWithDrifts, drifts) =
    AccuracyWithDrifts(clf, detector, dataFrame, trainingSamples)
DrawChart(accuracyWithoutDrifts, accuracyWithDrifts, drifts)
```

The run of the program above produces the following information:

```
Classifier name: RandomForestClassifier(random_state=0)
Drift detector name: DDM(callbacks=[],
config=DDMConfig(min_num_instances=30,
warning_level=2.0, drift_level=3.0))
Benchmark name: spam.arff
Accuracy without drifts = 0.276
Accuracy with drifts and retraining classifier = 0.533
Number of drifts detected = 1
```

Table 1: The average classification accuracy (ACC), chunk size, and number of detected drifts of artificial data stream processing for various drift detectors.

Dataset	DDM	EDDM	KSWIN	P-H	STEPD	ADWIN
Abrupt 5	0.631	0.633	0.631	0.631	0.852	0.844
	2300/0	2300/5	2300/0	2300/0	2300/25	2300/6
Abrupt 10	0.744	0.748	0.744	0.744	0.840	0.837
	2300/0	2300/10	2300/0	2300/0	2300/27	2300/5
Gradual 5	0.631	0.635	0.631	0.631	0.844	0.826
	2300/0	2300/10	2300/0	2300/0	2300/28	2300/6
Gradual 10	0.744	0.751	0.744	0.744	0.839	0.828
	2300/0	2300/6	2300/0	2300/0	2300/27	2300/5
Recurring 5	0.802	0.804	0.804	0.806	0.866	0.861
	2300/1	2300/0	2300/0	2300/1	2300/25	2300/3
Recurring 10	0.803	0.799	0.799	0.799	0.865	0.858
	2300/1	2300/0	2300/0	2300/1	2300/26	2300/3
Incremental 5	0.719	0.737	0.497	0.573	0.888	0.868
	2300/2	2300/9	2300/0	2300/1	2300/27	2300/10
Incremental 10	0.731	0.795	0.686	0.730	0.868	0.847
	2300/1	2300/6	2300/0	2300/1	2300/27	2300/7

P-H - Page-Hinkley. Number of features 20. Noise level 1%.

Using publicly available, open-source software, one can quickly conduct experiments on various detectors and their cooperation with multiple classifiers. Our experiment used a Random Forest classifier, the effectiveness (ACC) of which was checked after the selected detector signaled the drift.

3.2. Artificially generated data experiments

Artificial data has been generated using MOA tool (<https://moa.cms.waikato.ac.nz/>). This free and open-source tool is designed to experiment with data stream mining. Datasets with four types of concept drifts were generated: abrupt drift, gradual drift, recurring abrupt drift, and incremental drift.

The experimental results on artificial data are summarized in Table 1. The first column describes the names of the data streams, and the remaining ones describe classification accuracy, window size, and number of detected drifts for various drift detectors (DDM, EDDM, KSWIN, Page-Hinkley, STEPD, ADWIN). Rows are grouped in twos of the same type of drift (abrupt, gradual, recurring, and incremental), but a different number of drifting features (5 or 10).

3.3. Real data experiments

The real data were downloaded from the UCI Machine Learning Repository (<https://archive.ics.uci.edu/ml/index.php>). The repository is open, and the relevant data can be freely retrieved. Additionally, some benchmarks are described in the work [12].

Figures 5 and 6 show examples of graphs of classification accuracy in the function of sample number: (a) without drift detection and classifier re-training, and (b) with drift detection and classifier re-training. The visualized data comes from Spam.arff and Phishing.arff files and the classifier used was Random Forest. These graphs were obtained using the DrawChart() function from the above code.

The experimental results on real data are summarized in Table 2. The first column of Table 2 describes the names of the data streams. The first three rows of the Table concern real data with artificially induced drift, whereas the remaining ones contain descriptions of real data with natural drifts.

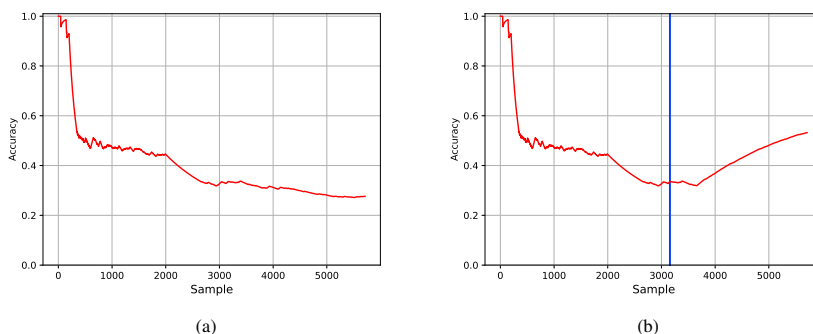


Fig. 5: (a) Accuracy without drifts detection and (b) accuracy with drifts detection (blue line) and with retraining Random Forest classifier on Spam.arff data file (DDM detector).

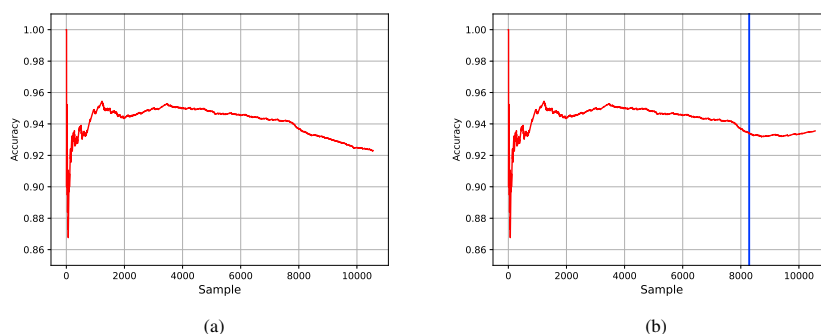


Fig. 6: (a) Accuracy without drifts detection and (b) accuracy with drifts detection (blue line) and with retraining Random Forest classifier on Phishing.arff data file (ADWIN detector).

4. Conclusions

The presented repository is a refined collection of open-source libraries and software. The package allows users to generate artificial data with various characteristics, containing various types of drifts, various numbers of features, and instances with the ability to add noise to the data. The idea of this repository is to familiarize the user with the classification of data streams and the detection of hidden drifts in the data stream. It is a toolset for users interested in machine learning and data analysis with hidden drifts.

Even those simple experiments, whose results are presented in Table 1 and Table 2, show that the efficiency of drift detection largely varies for different data sets and different drift detectors. For example, the KSWIN, has not detected a single drift in any of the artificially generated data, but on the other hand, EDDM has detected multiple drifts in Abrupt 5 and Abrupt 10 datasets with only one abrupt drift. This proves the importance of choosing the right drift detector for the particular data stream, and the choice is not trivial.

For this reason, software tools are needed to research various data drift detectors and compare their effectiveness reliably. The solution proposed in this article enables such research. The open-source development environment can be downloaded from the shared GitHub repository. The article presents the capabilities of the proposed software both in numerical research and generating performance charts of classifiers used in drift detectors. Graphs can be obtained for classifiers without drift detection and classifier training procedures after drift detection.

Our future work investigates how different drift forms affect classification quality, including new detectors based on feature drift. Additionally, our plans include examining the performance of unsupervised and supervised drift detectors in various class-label recognition scenarios. Additionally, we are trying to develop classifiers that can cope with drift occurring in many classes simultaneously.

Table 2: The average classification accuracy (ACC) of real data stream processing for various drift detectors. Chunk sizes and detected drifts are also stated.

Dataset	DDM	EDDM	KSWIN	P-H	STEPD	ADWIN
Digits08	0.987 150/0	0.985 150/2	0.987 150/0	0.987 150/0	0.987 150/0	0.987 150/0
Digits17	0.989 150/0	0.989 150/0	0.989 150/0	0.989 150/0	0.990 150/3	0.989 150/0
Wine	1.000 500/0	1.000 500/0	1.000 500/0	1.000 500/0	1.000 500/11	1.000 500/0
Spam	0.532 500/1	0.898 500/2	0.882 500/2	0.276 500/0	0.883 500/8	0.872 500/2
Phishing	0.923 500/0	0.921 500/5	0.923 500/0	0.923 500/0	0.935 500/8	0.936 500/1
CovType	0.715 2500/9	0.750 2500/14	0.861 2500/49	0.612 2500/3	0.885 2500/81	0.866 2500/51
Electricity	0.755 2500/5	0.766 2500/17	0.745 2500/13	0.728 2500/0	0.765 2500/16	0.757 2500/12
Pokerhand	0.707 A/0	0.708 A/8	0.711 A/1	0.711 A/1	0.710 A/16	0.707 A/0
Insect	0.494 2900/2	0.491 2900/10	0.412 2900/3	0.567 2900/1	0.568 2900/12	0.536 2900/7

P-H - Page-Hinkley. A = 5.5×10^4 .

References

- [1] Adams, J.N., van Zelst, S.J., Rose, T., van der Aalst, W.M., 2023. Explainable concept drift in process mining. *Information Systems* 114, 102177.
- [2] Agrahari, S., Singh, A.K., 2022. Concept drift detection in data stream mining : A literature review. *Journal of King Saud University - Computer and Information Sciences* 34, 9523–9540.
- [3] Baena-Garcia, M., Campo-Avila, J., Bifet, A., Gavalda, R., Morales-Bueno, R., 2006. Early drift detection. *Advances in Artificial Intelligence, Lecture Notes Artificial Intelligence* 3171, 286–295.
- [4] Bayram, F., Ahmed, B.S., Kassler, A., 2022. From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems* 245, 108632.
- [5] Bifet, A., Gavalda, R., 2007. Learning from time-changing data with adaptive windowing. *Proceedings of the 7th SIAM International Conference on Data Mining*, 443–448.
- [6] Gama, J., Medas, P., Castillo, G., Rodrigues, P., 2004. Learning with drift detection, 286–295.
- [7] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A., 2014. A survey on concept drift adaptation. *ACM Comp. Surveys* 46, 1–37.
- [8] Gonçalves, P.M., de Carvalho Santos, S.G., Barros, R.S., Vieira, D.C., 2014. A comparative study on concept drift detectors. *Expert Systems with Applications* 41, 8144–8156.
- [9] Jin, C., Feng, Y., Li, F., 2023. Concept drift detection based on decision distribution in inconsistent information system. *Knowledge-Based Systems* 279, 110934.
- [10] Palli, A.S., Jaafar, J., Gomes, H.M., Hashmani, M.A., Gilal, A.R., 2022. An experimental analysis of drift detection methods on multi-class imbalanced data streams. *Applied Sciences* 12, 11688.
- [11] Shao, J., Ahmadi, Z., Kramer, S., 2014. Prototype-based learning on concept-drifting data streams, 412–421.
- [12] Souza, V.M.A., dos Reis, D.M., Maletzke, A.G., Batista, G.E.A.P.A., 2020. Challenges in benchmarking stream learning algorithms with real-world data. *Data Min. Knowl. Discov.* 34, 1805–1858.
- [13] Suárez-Cetrulo, A.L., Quintana, D., Cervantes, A., 2023. A survey on machine learning for recurring concept drifting data streams. *Expert Systems with Applications* 213, 118934.
- [14] Webb, G.I., Hyde, R., Cao, H., Nguyen, H.L., Petitjean, F., 2016. Characterizing concept drift. *Data Mining and Knowledge Discovery* 30, 964–994.
- [15] Yu, H., Liu, W., Lu, J., Wen, Y., Luo, X., Zhang, G., 2023. Detecting group concept drift from multiple data streams. *Pattern Recognition* 134, 109113.
- [16] Yu, H., Zhang, Q., Liu, T., Lu, J., Wen, Y., Zhang, G., 2022. Meta-add: A meta-learning based pre-trained model for concept drift active detection. *Information Sciences* 608, 996–1009.